



*“All have their worth,” said Yavanna,
“and each contributes to the worth of the others”.*

J.R.R. Tolkien, *The Silmarillion*

CMPUT 365

Introduction to RL

Plan

- Dynamic programming
 - Finally, a solution method (albeit limited)!

Reminder (1 of 2)

You **should be enrolled in the private session** we created in Coursera for CMPUT 365.

I **cannot** use marks from the public repository for your course marks.

You **need** to **check, every time**, if you are in the private session and if you are submitting quizzes and assignments to the private section.

The deadlines in the public session **do not align** with the deadlines in Coursera.

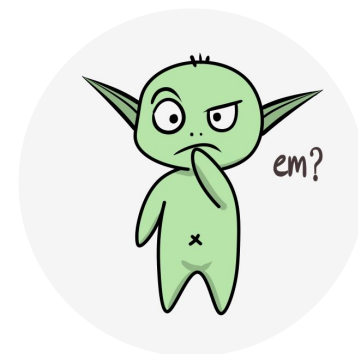
If you have any questions or concerns, **talk with the TAs** or email us
`cmput365@ualberta.ca`.

Reminder (2 of 2)

The midterm is next week!

- It will be written here, in the classroom (50 minutes).
- I need you to bring some piece of ID.
- It'll have multiple-choice (see Coursera) and open-ended questions (see Worksheets)
 - I don't know if we will have time to do a practice midterm, but it doesn't matter, if we do it, it will be a mix and match of Coursera and Worksheet questions. You don't need me for that.
- It is closed-book, no calculators are allowed.
- **There won't be a formula sheet.**
- Previous years:
 - 2023: Mean: 68%, Max: 100%
 - 2024: Mean: 65%, Max: 100%

Please, interrupt me at any time!



Dynamic Programming – Why?

- “DP provides an essential foundation for the understanding of the methods presented in the rest of this book”.
- ... but “classical DP algorithms are of limited utility in reinforcement learning both because of their assumption of a perfect model and because of their great computational expense”.
- “all of these [RL] methods can be viewed as attempts to achieve much the same effect as DP, only with less computation and without assuming a perfect model of the environment”.

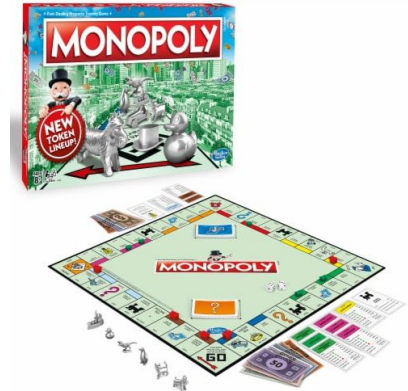
Models and Planning

- How should we think about $p(s', r \mid s, a)$? **It is a model. It tells us everything that is possible and impossible to happen (and their probability!)**
- Is dynamic programming different from what we did in bandits?

Figuring out how to act

Imagine the universe consists of you playing Monopoly against a computer. Your goal is to win the game.

There are two ways you can do so:

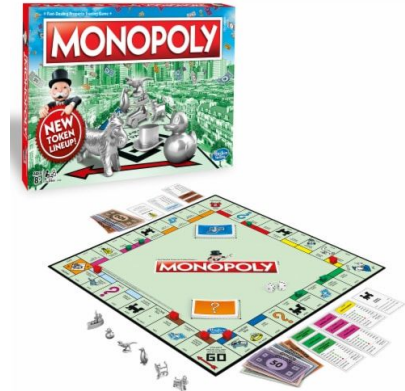


Figuring out how to act

Imagine the universe consists of you playing Monopoly against a computer. Your goal is to win the game.

There are two ways you can do so:

1. **Trial and error learning.** Play against it over and over, figure out the game rules and the computer's strategy.



Figuring out how to act

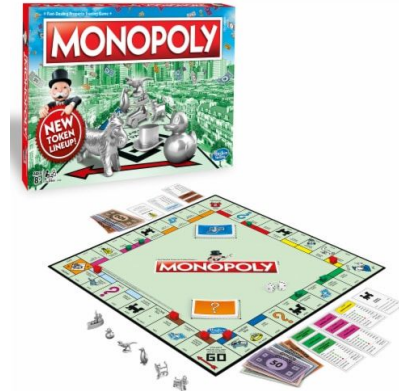
Imagine the universe consists of you playing Monopoly against a computer. Your goal is to win the game.

There are two ways you can do so:

1. **Trial and error learning.** Play against it over and over, figure out the game rules and the computer's strategy.
2. **Planning.** You could be given access to the game's rulebook as well as the code implementing the AI playing against you. You would then sit and there and **think** about how to win. You could **reason** about the rules and the AI, and **plan** how to win.

There's no
interaction!

The game's rulebook and the code implementing
the AI would allow you to compute $p(s', r | s, a)$.



Key Idea Behind Dynamic Programming

“To use value functions to organize and structure the search for good policies.”

*We use the same equations as before, but we replace an $=$ by a $\leftarrow$, that's it
(we turn Bellman equations into assignments).*

There's lots to decide

- What should we compute? v_π , q_π , v_* , q_* , π^* ?
- How should we select states to imagine about? And in what order?
- How much computation do we need to figure out the optimal policy, π^* , using the function p : $\mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$?
- How many times do we need to iterate over this imagining / planning process?

Obtaining value functions and π^ from π and p (with no interaction) is called*
Dynamic Programming.

Policy Evaluation (Prediction)

Given a policy and an MDP, what's the corresponding value function?

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$



$$\begin{aligned} v_{k+1}(s) &\doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')] \end{aligned}$$

expected update 

Policy Evaluation (Prediction)

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

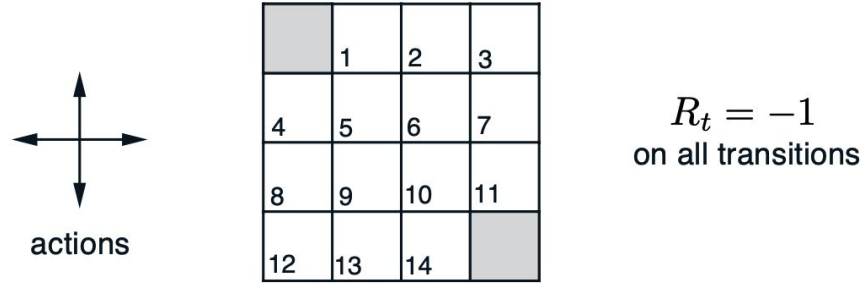
$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

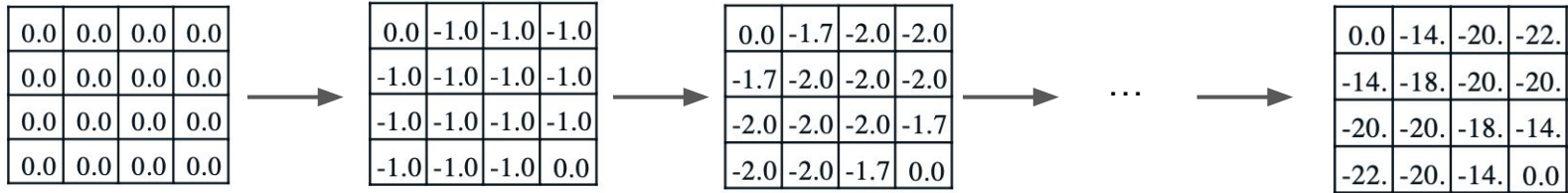
until $\Delta < \theta$

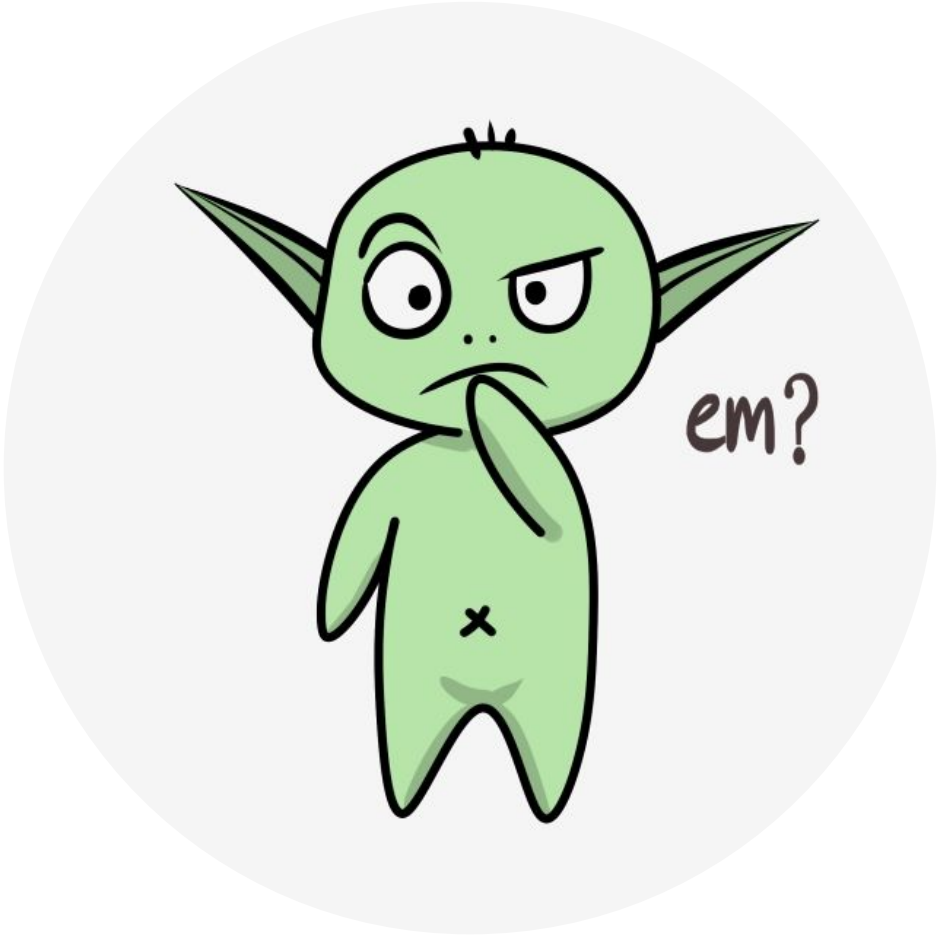
“in-place”
update

Policy Evaluation – Example



v_k for the
random policy





Policy Improvement

Given a value function for a policy π , how can we get a better policy π' ?

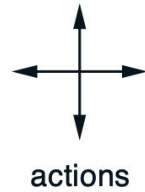
We already know how good policy π is, what if we acted differently now? What if instead of selecting action $\pi(s)$ we selected action $a \neq \pi(s)$, but then we followed π ?

We know the value of doing that!

$$\begin{aligned}
 q_{\pi}(s, a) &\doteq \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a] \\
 &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')].
 \end{aligned}$$

**If this new action is
better, in general
this new policy is
better overall**

Policy Improvement – Intuition



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

v_k for the
random policy

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



	←	↕	↕
↑	↕	↕	↕
↕	↕	↕	↓
↕	↕	→	

Policy Improvement Theorem

That this is true is a special case of a general result called the *policy improvement theorem*. Let π and π' be any pair of deterministic policies such that, for all $s \in \mathcal{S}$,

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s). \quad (4.7)$$

Then the policy π' must be as good as, or better than, π . That is, it must obtain greater or equal expected return from all states $s \in \mathcal{S}$:

$$v_{\pi'}(s) \geq v_{\pi}(s). \quad (4.8)$$

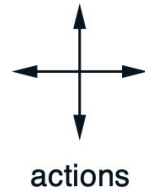
A more aggressive update

Instead of doing it for a particular action in a single state, we can consider changes at *all* states and to *all* possible actions.

$$\begin{aligned}\pi'(s) &\doteq \arg\max_a q_\pi(s, a) \\ &= \arg\max_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \arg\max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')],\end{aligned}$$

This is called *policy improvement*. And eventually it converges to the optimal policy.

Policy Improvement – Intuition



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

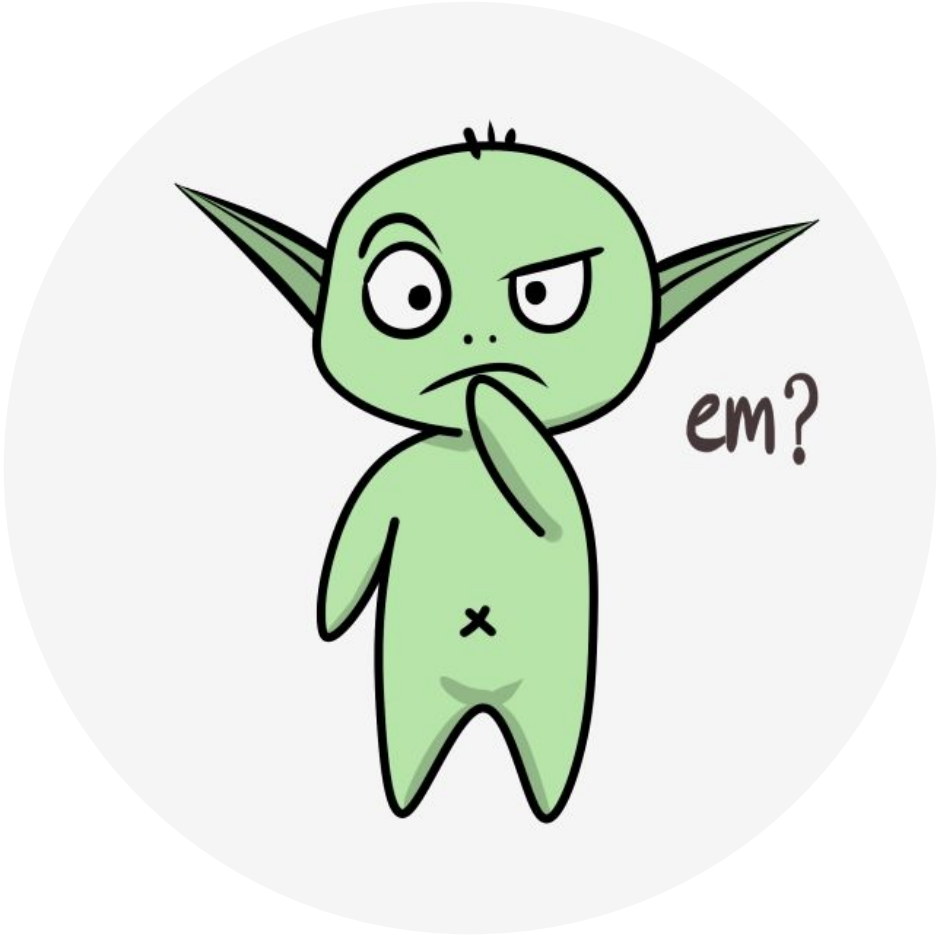
v_k for the
random policy

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



	←	←	↕
↑	↖	↕	↓
↑	↕	↘	↓
↕	→	→	



Policy Iteration: Interleaving Policy Eval. and Improvement

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow true$

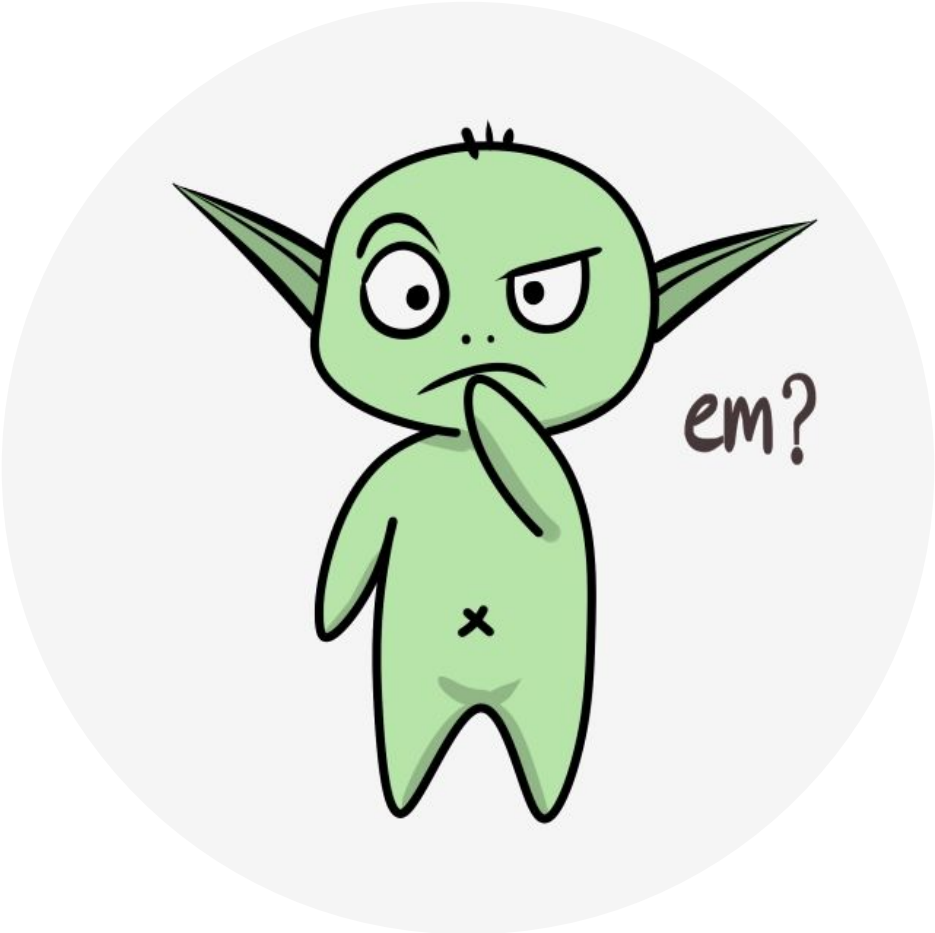
For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow false$

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2



Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
 Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

| $\Delta \leftarrow 0$

| Loop for each $s \in \mathcal{S}$:

| $v \leftarrow V(s)$

| $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

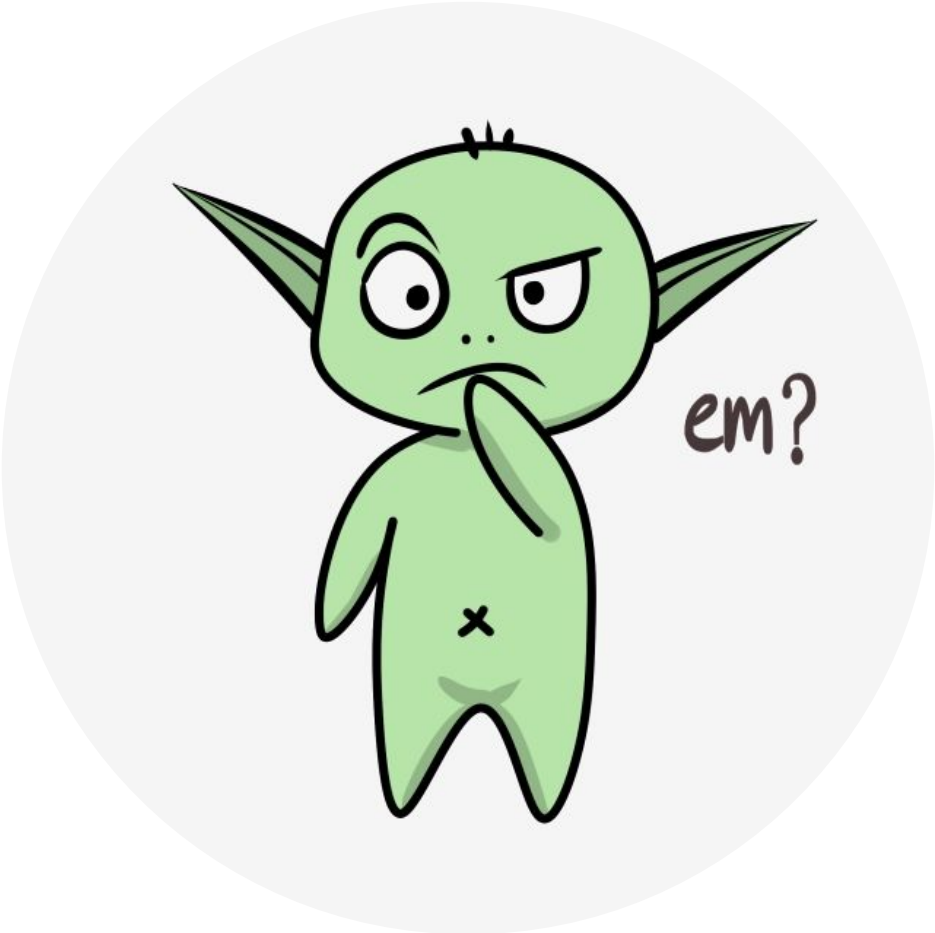
until $\Delta < \theta$

It doesn't need to be so synchronous

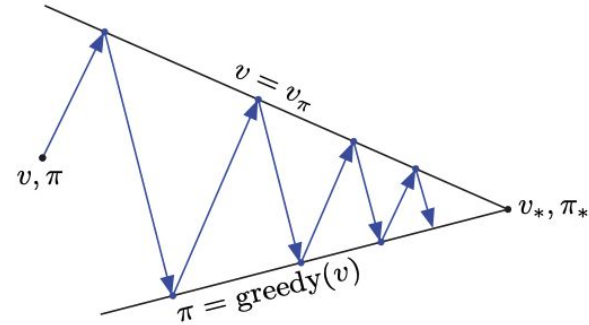
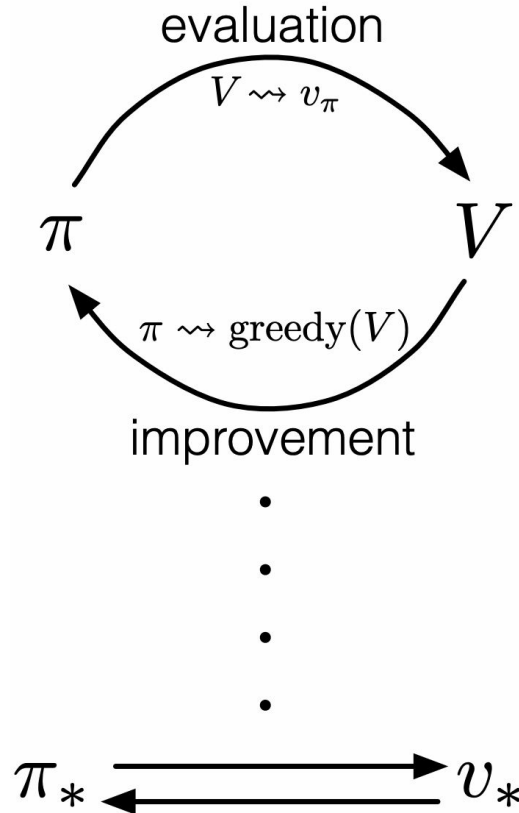
We just turned the Bellman optimality equation into an update rule!

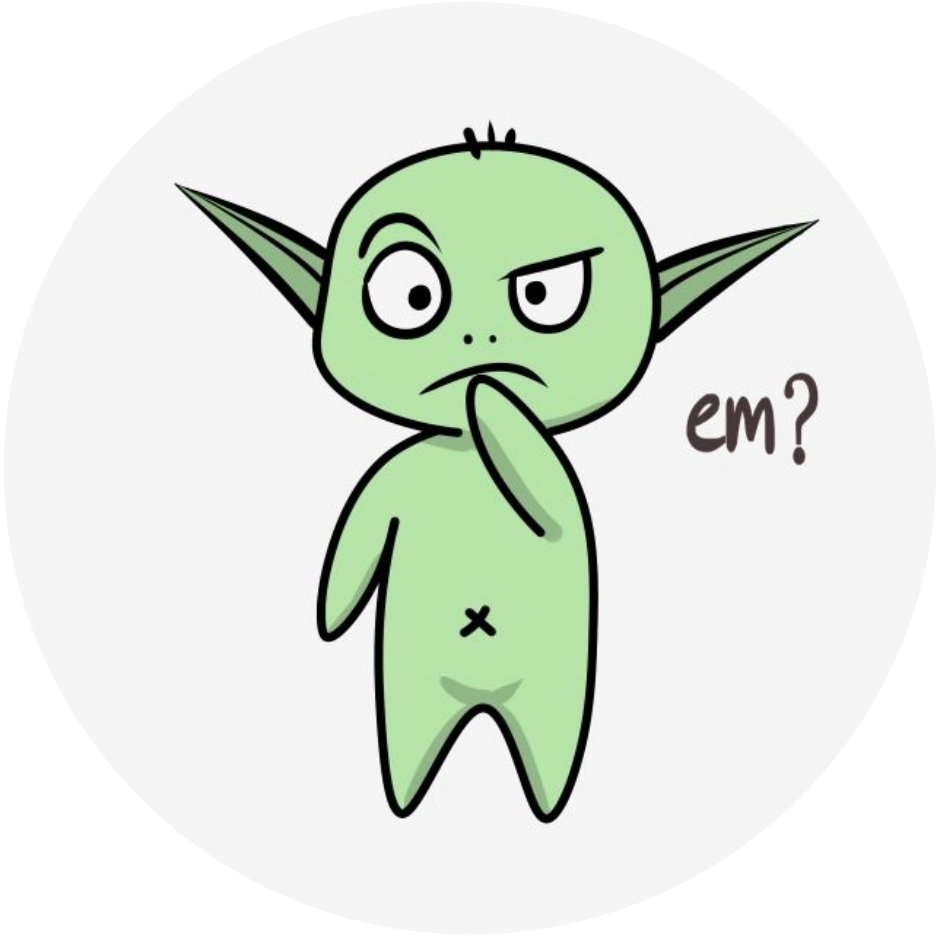
Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$



Generalized Policy Iteration





Next class

- What **I** plan to do: Finish discussing Chapter 4 on Dynamic Programming
- What I recommend **YOU** to do for next class:
 - Complete the assigned reading: Chapter 4, §4.1-§4.4 (pp. 73-84); §4.6-§4.7 (pp. 86-89)
 - Complete Practice quiz and Progr. assignment (Optimal policies with dynamic programming)
 - It is due on Monday!
- Next classes:
 - Monday: Dynamic Programming
 - Wednesday: General Overview and Q&A
 - **Friday: Midterm**